

Neural Networks & Activation Functions

Sattam Altwaim

sattam.tuuaim@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 3, Day 18, Self-study

6 October 2026

1. Where This Fits
2. Learning Outcomes
3. Activation Functions in Depth
4. What a ReLU Network Computes
5. Output Layers and Their Losses
6. Olympiad Practice
7. Check Yourself
8. Limitations and What Comes Next
9. Summary
10. References

- ▶ **In the lecture (Day 15):** a neuron is a line plus a bend, the bend is the activation function, a network stacks layers of neurons, and the chain rule carries the slope back through every layer. Day 17 added that deep sigmoid networks do not train without BatchNorm, and that dials such as dropout change what a unit sees.
- ▶ **This deck adds:** the slopes of the three activations with numbers and what happens to them across five layers; what a “dead” unit is and how to find one; what a ReLU network actually computes (a sum of hinges, drawn on real cars); width against depth measured; and the two output-layer mistakes that cost the most points.
- ▶ **How to use it:** every frame stands alone and every number was measured. Read a frame, type its code into a notebook, change one number, and compare with the frame. The self-check at the end tells you whether to reread.

Where the simple version breaks

- ▶ Day 15 said “ReLU inside, sigmoid or softmax at the output” and moved on. Day 17 then showed five sigmoid layers that never left a loss of 2.3. Why exactly, and would ReLU have saved them? The answer is a number: at the start of training the gradient reaching the first sigmoid layer is 4,900 times smaller than the one at the output layer; with ReLU the two are about the same size.
- ▶ “The network bends” is not a mechanism. With one input you can draw every piece: a ReLU network of 6 hidden units fits 3,598 used-car prices with MAE 39,017 against 41,724 for a straight line, and the picture shows which unit bends where.
- ▶ A network with a softmax inside it *trains*: the loss falls and the accuracy climbs. It is still wrong, and the gap only shows next to the right version: a training loss of 1.49 against 0.035 after eight epochs.

After this deck you can

read a network’s activations and output layer the way Day 16 taught you to read its loss curve: as a dashboard with a cause behind every symptom.

By the end of this self-study deck, you will be able to:

- ▶ **Compute** the slope of sigmoid, tanh and ReLU at a given z , and explain with numbers why slopes below 1 vanish through layers.
- ▶ **Diagnose** a dead ReLU unit and a saturated sigmoid from the numbers a network prints, and say what to change.
- ▶ **Draw** what a one-input ReLU network computes: a sum of hinges, one kink per hidden unit.
- ▶ **Choose** width and depth with a measured curve rather than a guess, and count the dials of the choice.
- ▶ **Pair** every output layer with its loss, and spot the two classic mistakes: a softmax inside the model, and an activation on a regression output.

Activations: **Activation Functions in Depth**

Backpropagation multiplies slopes (Day 15). The slope of the activation is one factor in every layer, so its size decides how much of the error signal survives the layer.

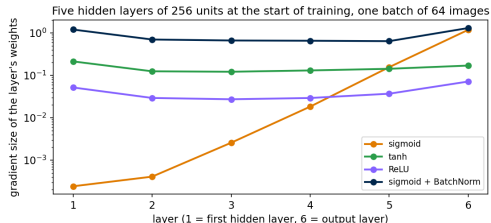
Activation	Slope formula	$z = 0$	$z = 2$	$z = 5$	$z = -5$
sigmoid	$\sigma(z) (1 - \sigma(z))$	0.25	0.105	0.0066	0.0066
tanh	$1 - \tanh^2(z)$	1	0.071	0.0002	0.0002
ReLU	1 if $z > 0$, else 0	0	1	1	0

- ▶ Sigmoid's slope is **never above 0.25**, and it is tiny once $|z| > 4$: the unit has **saturated**. tanh reaches 1 at $z = 0$ but saturates just as hard.
- ▶ ReLU's slope is exactly 1 on the whole right side: the signal passes through unchanged, however large z is. On the left it is exactly 0: nothing passes.
- ▶ Check one by hand: $\sigma(2) = 0.881$, so the slope is $0.881 \cdot 0.119 = 0.105$. At $z = 5$: $\sigma(5) = 0.9933$, slope $0.9933 \cdot 0.0067 = 0.0066$.

Five sigmoid layers, by hand

The error signal reaching layer 1 is multiplied by one sigmoid slope per layer on the way back. Even in the best case every slope is 0.25, so the signal arrives $0.25^5 = 0.00098$ times as large: a thousand times smaller before any weight is counted. Real units are not all at $z = 0$, so the factor is far below that.

- With ReLU the best-case factor is $1^5 = 1$: whatever reaches the last layer reaches the first.



Source: 784-256-256-256-256-10 networks at initialisation, gradient of each layer's weights on one batch of 64 MNIST images; `scripts/day18_activations_figures.py`.

Measured: with sigmoid the first layer's gradient is 4,900 times smaller than the output layer's; with tanh, ReLU, or sigmoid plus BatchNorm the six layers stay within a factor of three. BatchNorm (Day 17) keeps every z near 0, where the sigmoid slope is at its 0.25 maximum.

```
z = torch.tensor([0., 2., 5., 50.])
s = torch.sigmoid(z)
print(s)           # 0.50 0.88 0.993 1.0
print(s * (1 - s)) # 0.25 0.105 0.0066 0
print(torch.tanh(z)) # 0.00 0.96 1.000 1.0
print(torch.relu(z)) # 0. 2. 5. 50.
```

At $z = 50$ the sigmoid's slope is exactly 0 in `float32`: the unit is frozen and no step will unfreeze it.

- ▶ Where do z values of 50 come from? From **unscaled inputs**: a pixel of 255 or a mileage of 200,000 times a weight of 0.1 is already far outside the live zone.
- ▶ That is the mechanism behind Day 15's "scale first": standardised inputs (mean 0, spread 1) keep the first layer's z within a few units of 0.
- ▶ For deeper layers, BatchNorm does the same job after every layer. For ReLU the live zone is the whole right side: one more reason it is the default.

Definition

A ReLU unit is **dead** when its $z = \mathbf{w} \cdot \mathbf{x} + b$ is negative for every input. Its output and its slope are always 0, so no gradient reaches its weights: it stays dead.

```
model.eval()
with torch.no_grad():
    z = model[0](X_val)
    on = (z > 0).float().mean(0)
dead = (on == 0).sum().item()
print(dead, "dead units of", len(on))
```

z is layer 1 before its ReLU; on is the share of rows on which each unit fires. A unit that fires on none is dead.

- ▶ How units die: one big update (a large rate, an outlier batch) pushes b far negative; the unit never fires again.
- ▶ Measured on 784-128-10 after three epochs of SGD: rate 0.05 kills no unit (accuracy 0.944); 0.5 kills 21 of 128 (0.948); 1.0 kills 90 of 128 and the accuracy collapses to 0.546.
- ▶ The fix is upstream: a smaller rate, scaled inputs, BatchNorm. **Leaky ReLU** (`nn.LeakyReLU()`, slope 0.01 on the left) lets a unit come back.

Activations: **What a ReLU Network Computes**

With a single input x , hidden unit j computes $h_j = \text{ReLU}(w_j x + b_j)$ and the output adds $v_j h_j$. The function $v_j \text{ReLU}(w_j x + b_j)$ is a **hinge**: 0 on one side of the kink $x = -b_j/w_j$, a line of slope $v_j w_j$ on the other.

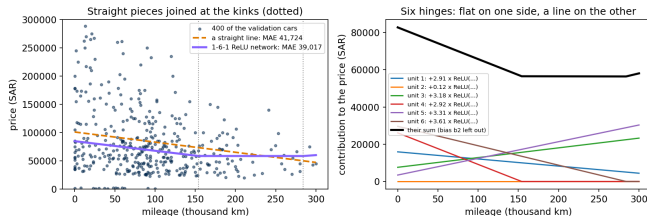
With numbers

$w = 1$, $b = -2$, $v = 3$: the hinge is $3 \cdot \text{ReLU}(x - 2)$. At $x = 0$: $\text{ReLU}(-2) = 0$, contribution 0. At $x = 2$: still 0 (the kink). At $x = 4$: $\text{ReLU}(2) = 2$, contribution 6. At $x = 5$: 9. Slope 3 to the right of 2, flat to the left.

With $w = -1$, $b = 2$, $v = 3$: $3 \cdot \text{ReLU}(2 - x)$, the mirror image: flat to the right of 2, slope -3 to the left.

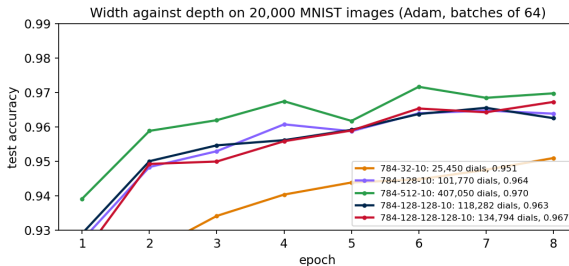
- ▶ The output layer adds the hinges and a bias: $\hat{y} = b_2 + \sum_j v_j \text{ReLU}(w_j x + b_j)$. A sum of straight pieces joined at kinks is a **piecewise linear** curve: with m hidden units, at most m kinks.
- ▶ Training moves the kinks (through w_j, b_j) and the slopes (through v_j). A ReLU network never draws a true curve, only straight pieces, as many as it has units.

Six Hinges on 3,598 Cars



Source: 3,598 listings from 2015 on, at most 300,000 km, Kaggle turkibintalib/saudi-arabia-used-cars-dataset, 80/20 split; 1-6-1 ReLU network, L1 loss, Adam; scripts/day18_activations_figures.py.

- ▶ Left: the fit is straight pieces: a drop over the first 150,000 km, then nearly flat. It beats the line by 2,700 SAR of MAE (39,017 against 41,724) with no hand-made x^2 column: the bends are the units.
- ▶ Right: units 4 and 6 bend inside the range: a slope down, then zero past kinks near 153,000 and 283,000 km (dotted). Units 1, 3 and 5 kink outside the range, so here they are lines; unit 2 is almost zero. Their sum plus the output bias is the purple curve.



Source: 20,000 MNIST training images, 10,000 test images, Adam at 0.001, batches of 64, eight epochs, the same seed for every shape; scripts/day18_activations_figures.py.

- ▶ Wider helps, with diminishing returns: 32 units 0.951, 128 units 0.964, 512 units 0.970, for 25,450, 101,770 and 407,050 dials. Four times the dials buys 0.6 points.
- ▶ Deeper buys little here: two layers of 128 score 0.963 and three score 0.967, no better than one wide layer. On a flattened image a single hidden layer already bends enough; depth pays when the layers can share structure (Week 4's convolutions).

dials of a layer = inputs $\times$ units + units

dials of the network = the sum over its layers

Shape	Arithmetic	Dials
784-32-10	$784 \cdot 32 + 32 + 32 \cdot 10 + 10$	25,450
784-128-10	$784 \cdot 128 + 128 + 128 \cdot 10 + 10$	101,770
784-512-10	$784 \cdot 512 + 512 + 512 \cdot 10 + 10$	407,050
784-128-128-10	$101,770 - 1,290 + 128 \cdot 128 + 128 + 1,290$	118,282

- ▶ The first layer owns almost everything: 784 inputs make every one of its units expensive. A second hidden layer of 128 costs only 16,512 dials, a sixth of a 512-wide first layer.
- ▶ Rule for a table or flattened image: start at inputs-128-outputs; go wider only if the training loss stays high (underfitting), deeper only if the task has structure one layer cannot bend into. Judge both on the validation curve.
- ▶ `sum(p.numel() for p in model.parameters())` does the count; write the arithmetic once by hand so the number means something.

Activations: **Output Layers and Their Losses**

For two classes, softmax of the logits (z_1, z_2) gives class 1 the probability

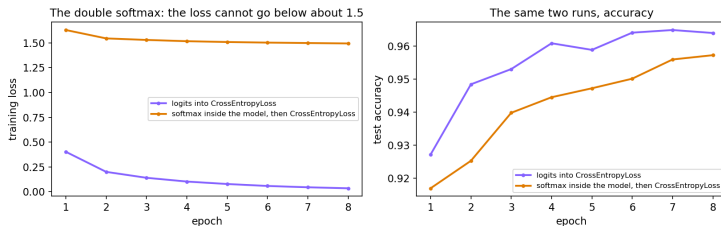
$$\frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{1}{1 + e^{-(z_1 - z_2)}} = \sigma(z_1 - z_2)$$

With numbers

Logits $(2, 0)$: $e^2 = 7.389$, $e^0 = 1$, softmax $= (7.389/8.389, 1/8.389) = (0.881, 0.119)$.
One logit $z = 2 - 0 = 2$: $\sigma(2) = 0.881$. The same number, so a yes/no task needs one output neuron, not two.

- ▶ Only differences between logits matter to softmax: adding 100 to every logit changes nothing. That is why a network's raw outputs can sit anywhere, and why the loss, not the model, should turn them into probabilities.
- ▶ `BCEWithLogitsLoss` on one output and `CrossEntropyLoss` on two outputs train the same model; the first is simpler and is what Day 16's labs use.

The Double Softmax, Measured



Source: 784-128-10 on 20,000 MNIST images, Adam at 0.001, eight epochs; the only difference is an `nn.Softmax(dim=1)` as the model's last layer; `scripts/day18_activations_figures.py`.

- ▶ The double softmax trains: accuracy 0.957 after eight epochs against 0.964 for the right model. That is why it goes unnoticed. Its training loss, 1.49 against 0.035, is the tell: a probability vector pushed through a second softmax can never be confident, so the loss has a floor near 1.5.
- ▶ On a harder task the gap in accuracy grows and the model stops improving early. Rule: the last layer is `nn.Linear`; `CrossEntropyLoss` does the softmax.

Target	Last layer	Loss	To read a prediction
a number	Linear(h, 1), nothing after	L1Loss or MSELoss	the output itself
yes / no	Linear(h, 1), nothing after	BCEWithLogitsLoss	torch.sigmoid(out)
one of k	Linear(h, k), nothing after	CrossEntropyLoss	softmax(out, 1) or argmax

```
model = nn.Sequential(nn.Linear(23, 64), nn.ReLU(),  
                      nn.Linear(64, 1))      # a price: stop here  
pred = model(X)                             # dollars, any value  
probs = torch.sigmoid(model_yes_no(X))      # only to read a yes/no
```

- ▶ The rule has no exceptions in this course: **the last layer is linear**. The loss supplies the sigmoid or the softmax during training; you supply it, by hand, only to read probabilities.
- ▶ The two classic mistakes: a softmax or sigmoid *inside* the model before the loss (the previous frame), and an activation on a regression output (the practice task).

Activations: Olympiad Practice

Task (Day 08 and Day 16, revisited)

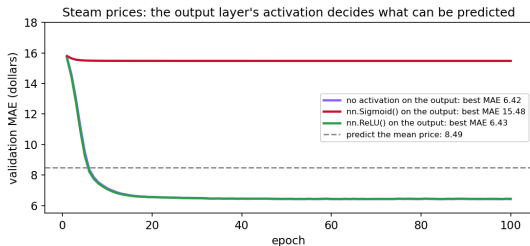
The Steam prices, scored by MAE (full points below 6.6). A teammate trains a network for 100 epochs with Day 17's recipe and the L1 loss, and the validation MAE never drops below 15. Predicting the mean price for every game scores 8.49. Here is the model.

```
# "the sigmoid keeps the output positive"
model = nn.Sequential(nn.Linear(23, 64), nn.ReLU(),
                      nn.Linear(64, 1), nn.Sigmoid())
```

- ▶ **Question 1:** without running anything, what is the largest price this model can predict, and what does that do to a game priced at 60 dollars?
- ▶ **Question 2:** the teammate replaces `nn.Sigmoid()` by `nn.ReLU()` “because prices are never negative”. Does that fix it, and what new risk does it bring?
- ▶ **Question 3:** which single line gives the right model, and how do you check the fix from the predictions alone, before looking at the MAE?

Adapted from the ONIA (Romanian AI Olympiad) problem-proposal example “Predicting Game Prices on the Steam Platform”, 2024/25; data from the Kaggle Steam Games data

set.



Source: Day 16's features, 2,400 / 600 split, 23-64-1 networks, Adam at 0.001, L1 loss, 100 epochs; scripts/day18.activations_figures.py.

1. A sigmoid outputs numbers between 0 and 1, so the largest price it can predict is 1 dollar. Measured: every prediction is 1.00 and the MAE is 15.48, worse than the mean's 8.49. A 60-dollar game alone costs 59 dollars of error.
2. `nn.ReLU()` works here (MAE 6.43, predictions 2.80 to 55.48) because prices are positive, but it gains nothing over no activation (6.42) and adds a risk: if the output's z goes negative for every game, the output is dead at 0 and never recovers.
3. Delete the activation: `nn.Linear(64, 1)` is the last layer. Check `model(X_val).min()` and `.max()`: the predictions must span the real prices (0 to about 70 dollars), not a corner of them.

Activations: **Check Yourself**

Answer before turning the page.

1. **Definition.** A hidden unit's z is -30 for every row of the data. What is its output and its slope (a) with a sigmoid, (b) with ReLU, (c) with Leaky ReLU, and which of the three units can still recover during training?
2. **Mechanism.** A one-input ReLU network has hidden units $(w, b, v) = (1, -1, 2)$ and $(1, -3, -2)$ and output bias 0. Compute $\hat{y}$ at $x = 0, 1, 2, 3, 4$ and describe the shape of the curve.
3. **Code.** What is wrong with this model for a task scored by accuracy over 10 classes, and how does it show up during training?

```
model = nn.Sequential(nn.Linear(784, 128), nn.Sigmoid(),  
                      nn.Linear(128, 10), nn.Softmax(dim=1))  
loss_fn = nn.CrossEntropyLoss()
```

1. (a) Sigmoid: output $\sigma(-30) \approx 10^{-13}$, slope $\approx 10^{-13}$: saturated, in practice frozen. (b) ReLU: output 0, slope 0: dead; no gradient reaches its weights, so it cannot recover. (c) Leaky ReLU: output -0.3 , slope 0.01: small, but not zero, so the weights still move and the unit can come back. Only (c) recovers on its own; (a) and (b) need the inputs or the rate fixed upstream.
2. Unit 1 is $2\text{ReLU}(x - 1)$: kink at 1, slope 2 to the right. Unit 2 is $-2\text{ReLU}(x - 3)$: kink at 3, slope -2 to the right. $\hat{y}$ at $x = 0, 1, 2, 3, 4$: 0, 0, 2, 4, 4. Flat until 1, rising with slope 2 until 3, flat again: the second hinge cancels the first past $x = 3$. Two units, two kinks, three straight pieces.
3. Two problems. The Softmax inside the model followed by CrossEntropyLoss is a double softmax: the loss cannot fall below about 1.5 and the accuracy climbs slowly (the measured frame). The sigmoid hidden layer trains, but more slowly than ReLU, and would stall in a deeper network. Fix: `nn.ReLU()` inside, no activation after the last Linear, and read probabilities with `softmax` only after training.

Activations: **Limitations & What Comes Next**

- ▶ **The hinge picture is exact only with one input.** With 784 inputs the kinks become hyperplanes and the pieces become regions you cannot draw; the mechanism is the same, the intuition is all you keep.
- ▶ **ReLU is a default, not a law.** Variants (Leaky ReLU, GELU) win on some tasks by a little; sigmoid and tanh still live inside recurrent networks (Week 6), where their bounded output is the point.
- ▶ **Width and depth were measured on one data set.** The ranking changes with the data: on 20,000 MNIST images a second layer bought almost nothing; on images read by convolutions (Week 4) depth is everything.
- ▶ **Dead units are a symptom, not a disease.** Counting them tells you a rate or a scaling is wrong; the count itself is not a dial.

The common mistake

Fixing a stalled network by swapping activations at random. Read the dashboard first: a loss at $\ln k$ from the start is a rate or a scaling problem, a loss near 1.5 on ten classes is a double softmax, a regression MAE stuck near the mean's is an output activation.

- ▶ **Day 19, self-study:** tensors, datasets and DataLoaders in depth: shapes, dtypes, custom datasets with transforms, and the loading speed that decides Week 4's image runs.
- ▶ **Day 20, self-study, Olympiad practice:** reading loss curves, the optimizer and regularisation dials, and the habits that win points on Friday.
- ▶ **Week 4:** the same neurons and the same activations, arranged as convolutions; the first thing you will check in every network is still its last layer and its loss.

Activations: **Summary**

- ▶ **Slopes:** sigmoid never above 0.25 and near 0 past $|z| = 4$, tanh the same past $|z| = 3$, ReLU exactly 1 or 0. Multiplied through five sigmoid layers, even the best case is 0.001: the vanishing gradient, measured as a 4,900-fold gap between the last and the first layer.
- ▶ **Saturated and dead units** come from unscaled inputs and large rates; the fixes are upstream (scaling, BatchNorm, a smaller rate), with Leaky ReLU as the activation that can recover.
- ▶ **A ReLU network is a sum of hinges:** one kink per hidden unit, straight pieces between; six units fit 3,598 used-car prices better than a line, and the picture shows which unit bends where.
- ▶ **Width and depth** are measured, not guessed: the first layer owns the dials, a second layer is cheap, and the validation curve decides.
- ▶ **The last layer is linear:** the loss supplies the sigmoid or softmax; a softmax inside the model or an activation on a regression output are the two mistakes to check first.

Activations: **References & Resources**

- [1] Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *AISTATS*. Vanishing gradients through saturating activations.
- [2] Nair, V. and Hinton, G. (2010). Rectified linear units improve restricted Boltzmann machines. *ICML*. ReLU.
- [3] Maas, A., Hannun, A. and Ng, A. (2013). Rectifier nonlinearities improve neural network acoustic models. *ICML workshop*. Leaky ReLU.
- [4] Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *NeurIPS* 32. `torch.nn` activations and losses. <https://pytorch.org/docs>
- [5] LeCun, Y., Cortes, C. and Burges, C. *The MNIST database of handwritten digits*, via OpenML (`mnist_784`); the Day 15 split for every MNIST run in this deck.
- [6] Bin Talib, T. *Saudi Arabia Used Cars Dataset*, Kaggle `turkibintalib/saudi-arabia-used-cars-dataset`; the 3,598 listings from 2015 on with at most 300,000 km, for the hinge figure.
- [7] ONIA (Romanian AI Olympiad), official problem-proposal example *Predicting Game Prices on the Steam Platform*, Olimpiada-AI, 2024/25. The practice task.

- [8] Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*, MIT Press, chapter 6: hidden units, output units, piecewise linear networks.
- [9] All plots: `slides-builder/scripts/day18_activations_figures.py` (PyTorch runs on real data; it also prints every number quoted in this deck).